

HxUI Toolkit

Davide Berti
Software Engineer
Developer Technical Services

HxUI Toolkit

- UI Primitives
- Features
- Hierarchy
- Examples
- Methodology



UI Primitives

- Containers
- Draw Callbacks
- Extended Lists
- Dialogs
- Tabs
- Fields
- Alerts
- Command Buttons
- Checkboxes
- Menus
- Scrollbars
- SpinButtons
- Pinwheels
- ProgressBars
- Lists
- Labels

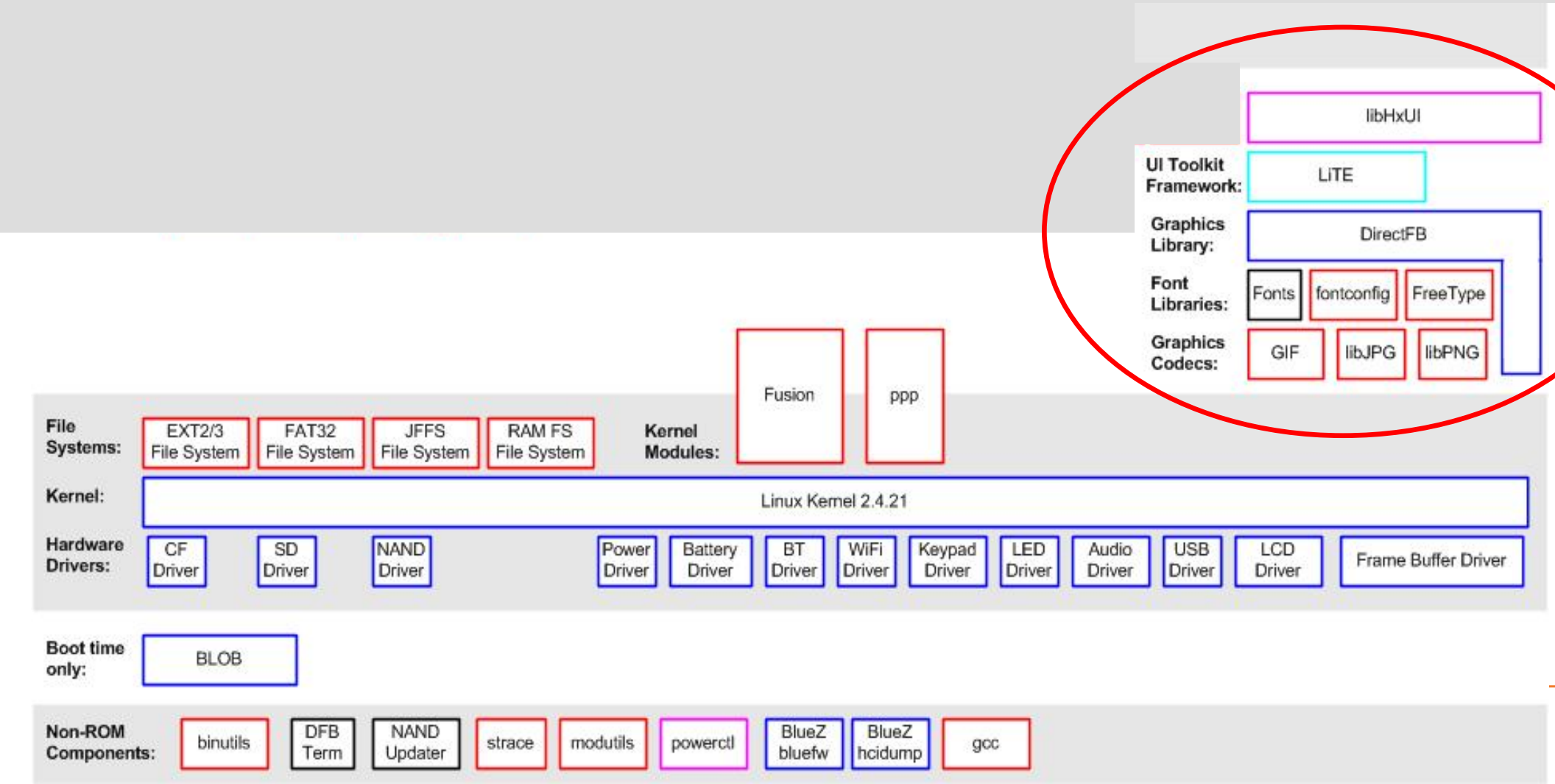
…



Features

- Clipboard Interface
  - Cut/Copy/Pase
- Tab Views
- HtmlView Container
  - Embed HTML in a container object
- Soft Keyboard
  - Software Keyboard avaiable via an API set
- Flexible Image Rendering
  - Inclusion of image data at compile time – Faster
  - Inclusion of image data at run-time
- Gadget Objects
  - Can be coupled to a Text Label or Image
- Flexible UI Primitive Construction
  - Via resource File
  - Via APIs




libHxUI
UI Toolkit Framework:
LiTE
Graphics Library:
DirectFB
Font Libraries:
Fonts
fontconfig
FreeType
Graphics Codecs:
GIF
libJPG
libPNG
Fusion
ppp
File Systems:
EXT2/3 File System
FAT32 File System
JFFS File System
RAM FS File System
Kernel Modules:
Kernel:
Linux Kernel 2.4.21
Hardware Drivers:
CF Driver
SD Driver
NAND Driver
Power Driver
Battery Driver
BT Driver
WiFi Driver
Keypad Driver
LED Driver
Audio Driver
USB Driver
LCD Driver
Frame Buffer Driver
Boot time only:
BLOB
Non-ROM Components:
binutils
DFB Term
NAND Updater
strace
modutils
powerctl
BlueZ bluefw
BlueZ hcidump
gcc

Coding to the Falcon API: Widgets…

Widget structs are opaque. Here is their structure:

DirectFB surface	LiTE LiteBox	HxUi HxWidget	HxUi HxCommandButton
Surface	Surface	Surface	Surface
	LiteBox	LiteBox	LiteBox
		HxWidget	HxWidget
			HxCommandButton

HxCommandButton could be cast as HxWidget and used in HxWidget calls.



hxgraphics.h

- HxResult HxGraphicsDrawLine (HxWidget_t *pWidget, int32_t x1, int32_t y1, int32_t x2, int32_t y2)
  - Draw a line.
- HxResult HxGraphicsDrawSeparator (HxWidget_t *pWidget, int32_t x1, int32_t y1, int32_t width)
  - Draw a separator.
- HxResult HxGraphicsDrawLines (HxWidget_t *pWidget, const HxSegment_t *pLines, uint32_t lineCount)
  - Draw an array of lines.
- HxResult HxGraphicsFillRectangle (HxWidget_t *pWidget, const HxRectangle_t *pRect)
  - Draw a filled rectangle.
- HxResult HxGraphicsDrawRectangle (HxWidget_t *pWidget, const HxRectangle_t *pRect)
  - Draw a rectangle.
- HxResult HxGraphicsFillTriangle (HxWidget_t *pWidget, int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3, int32_t y3)
  - Draw a filled triangle.
- HxResult HxGraphicsDrawTriangle (HxWidget_t *pWidget, int32_t x1, int32_t y1, int32_t x2, int32_t y2, int32_t x3, int32_t y3)
  - Draw a triangle.



hxgraphics.h

- HxResult HxGraphicsFillPolygon (HxWidget_t *pWidget, const HxPolygon_t *pPolygon)
  - Draw a filled polygon.
- HxResult HxGraphicsDrawPolygon (HxWidget_t *pWidget, const HxPolygon_t *pPolygon)
  - Draw a polygon.
- HxResult HxGraphicsFillSpans (HxWidget_t *pWidget, int32_t y, const HxSpan_t *spans, uint32_t spanCount)
  - Draw a filled array of spans.
- HxResult HxGraphicsDrawString (HxWidget_t *pWidget, HxFont_t *pFont, const char *pString, int32_t x, int32_t y)
  - Draw a text string.
- HxResult HxGraphicsDrawStringWithEllipsis (HxWidget_t *pWidget, HxFont_t *pFont, HxEllipsisType_t type, const char *pString, int32_t x, int32_t y, int32_t width)
  - Draw a text string width Ellipsis.
- HxResult HxGraphicsDrawStringWithFlags (HxWidget_t *pWidget, HxFont_t *pFont, const char *pString, int32_t x, int32_t y, HxTextFlags_t flags)
  - Draw a text string using flags.
- HxResult HxGraphicsBlit (HxImage_t *pSrcImage, HxImage_t *pDstImage, const HxRectangle_t *pSrcRect, int32_t x, int32_t y)
  - Blit an image.



```
static void DrawContainerWindow(HxWindow_t* pWindow, const
      HxRegion_t* pRegion,bool clear)
{
  HxResult result = HX_OK;
  HxRectangle_t            bounds;
  HxColor_t                        color = { 255, 255, 255, 255
     };

  result = HxWidgetGetBounds((HxWidget_t *)pWindow, &bounds);
  DDEBUG("Get Bounds result:%d\n", result);

  bounds.x = bounds.y = 0;

  result = HxGfxSetColor((HxWidget_t *)pWindow, &color);
  result = HxGfxFillRectangle ((HxWidget_t *)pWindow, &bounds);

  HX_ASSERT(HX_OK == result);
}
```

libhxui – hxdialog.h

Functions

- HxResult HxDialogNew (uint32_t id, const HxRectangle_t *pBounds, const char *pTitle, HxDialog_t **hDialog)
  - Create a dialog.
- HxResult HxDialogDelete (HxDialog_t *pDialog)
  - Close a dialog.
- HxResult HxDialogShow (HxDialog_t *pDialog, bool show)
  - Show or hide a dialog.
- HxResult HxDialogShowAndWait (HxDialog_t *pDialog)
  - Show or hide a dialog.
- HxResult HxDialogGetTitle (HxDialog_t *pDialog, char **const hText)
  - Get the title of the dialog.
- HxResult HxDialogSetTitle (HxDialog_t *pDialog, const char *pText)
  - Set the title of the dialog.
- HxResult HxDialogSetEscapeButton (HxDialog_t *pDialog, uint32_t buttonID)
  - Set the "escape" command button.



Widget Callbacks - Overridable

```
typedef int32_t(*) HxWidgetOnEnterFnPtr (HxWidget_t *pWidget,
  int32_t x, int32_t y)
```

- The OnEnter callback function is called when the cursor enters the bounds of a widget.

```
typedef int32_t(*) HxWidgetOnLeaveFnPtr (HxWidget_t *pWidget,
  int32_t x, int32_t y)
```

- The OnLeave callback function is called when the cursor exits the bounds of a widget.

```
typedef int32_t(*) HxWidgetOnMotionFnPtr (HxWidget_t *pWidget,
  int32_t x, int32_t y, HxMouseButtonMask_t buttons)
```

- The OnMotion callback function is called when the cursor moves. Minimize work done by motion callbacks because it is called frequently.

```
typedef int32_t(*) HxWidgetOnButtonDownFnPtr (HxWidget_t
  *pWidget, int32_t x, int32_t y, HxMouseButton_t button)
```

- The OnButtonDown callback function is called when the button down occurs within the bounds of the widget.



Overriding a Container's Draw Callback

- A gadget can contain one or more objects. The object definition includes a type, which can either by a system type (text or image), or a user type. The toolkit draws a system type using the widget Draw callback, while the application is responsible for drawing objects that are user types.
- Gadgets maintain state information
  - `kHxGadgetStateNormal`
  - `kHxGadgetStatePressed`
  - `kHxGadgetStateHilite`
  - `kHxGadgetStateDisabled`
  - `kHxGadgetStateCount`





```
result = HxWidgetSetCallback((HxWidget_t*)TheContainer,
                kHxWidgetDraw, DrawContainerWindow, NULL);

HX_EXIT_IF_ERROR(HX_OK == result, result, error);
```



```
static void DrawContainerWindow(HxWindow_t* pWindow, const
        HxRegion_t* pRegion,bool clear)
{
    HxResult                result = HX_OK;
    HxRectangle_t           bounds;
    HxColor_t               color = { 255, 255, 255, 255 };

    result = HxWidgetGetBounds((HxWidget_t *)pWindow, &bounds);
    DDEBUG("Get Bounds result:%d\n", result);

    bounds.x = bounds.y = 0;

    result = HxGfxSetColor((HxWidget_t *)pWindow, &color);
    result = HxGfxFillRectangle ((HxWidget_t *)pWindow, &bounds);

    HX_ASSERT(HX_OK == result);
}
```



Building a Gadget Object

- A gadget can contain one or more objects. The object definition includes a type, which can either by a system type (text or image), or a user type. The toolkit draws a system type using the widget Draw callback, while the application is responsible for drawing objects that are user types.
- Gadgets maintian state information
  - `kHxGadgetStateNormal`
  - `kHxGadgetStatePressed`
  - `kHxGadgetStateHilite`
  - `kHxGadgetStateDisabled`
  - `kHxGadgetStateCount`



```
HxImageDescription_t   *DescriptionGadgets[NUM_GADGETS];
HxGadgetObject_t       ObjArray[NUM_GADGETS];

HxResult LoadGadgets(void)
{
  int num = 0;
  char imagePath[IMAGE_BUFFER_SIZE];
  HxResult result = HX_OK;

  for(; num < NUM_GADGETS; num++)
  {
        result = HxImageDescriptionNewFromFile(imagePath,
                &DescriptionGadgets[num]);
        HX_EXIT_IF_ERROR( HX_OK == result, result, error );

        ObjArray[num].type = kHxGadgetObjImage;
        ObjArray[num].image.pImage = DescriptionGadgets[num];
        ObjArray[num].x = (num * (GADGET_WIDTH/NUM_GADGETS));
        ObjArray[num].y = 0;
        }
        ….
```





```
result = HxGadgetNew(GADGET_ID, (HxWidget_t*)AppWindow, &rect,
            NUM_GADGETS, ObjArray,
            kHxGadgetHandlesInput | kHxGadgetHasFrame, &Gadget);

HX_EXIT_IF_ERROR(HX_OK == result, result, error);

/* listen for gadget selected messages */
result = HxWidgetMapMessageCallback((HxWidget_t*)Gadget,
                kHxActionMessageClass, kHxGadgetSelected,
                GadgetCallback, Gadget);
```



Methodology

- Which widgets will I use to interact with the user?
- Which data structures/routines can I use that glib provides?
- Use the hxui xml file to prototype UI elements and localize your application
- Load images at compile time with the directfb-csource utility
- Use alpha-blended images
- Analyze performance with a release build that has been prelinked



Methodology

- Factor in the design paradigm of the Falcon
  - Instant on
  - Immediate application switching
  - Easy to Use
- Determine whether your application will use the HxUI library or communicate with DirectFB.
- Do I need a UI Toolkit or is the UI based on an internal specification?
- Do I want to internationalize my application?
- What information will need to be stored when an application is switched into the background? What information should be restored when an application is switched into the foreground?

